UNIVERSIDAD DE BUENOS AIRES
FACULTAD DE INGENIERÍA
Año 2024 – 2do Cuatrimestre

(TB067) REDES DE COMUNICACIONES

TRABAJO PRÁCTICO: Niveles de Aplicación y Transporte

Curso: Curso 3

9 de Octubre de 2024

Berni, Franco	110007	fberni@fi.uba.ar
Hirsch, Manuel	110221	mhirsch@.fi.uba.ar

I. RESEÑA MQTT

Esta sección se corresponde con la resolución del ejercicio 1.

El protocolo MQTT (Message Queuing Telemetry Transport) es un estándar de comunicación ligero basado en el modelo publish/subscribe. Fue diseñado para la conectividad de dispositivos en el contexto del Internet de las Cosas (IoT). Desarrollado inicialmente por IBM en 1999, MQTT ha evolucionado significativamente a lo largo de los años. Su especificación está regulada por la organización OASIS. Típicamente la conexión se realiza utilizando TCP/IP.

La versión más reciente, MQTT 5.0, fue lanzada en 2019. Esta versión introdujo mejoras sustanciales en escalabilidad y diagnóstico, tales como códigos de error y balanceo de carga. Sus predecesoras, MQTT 3.1.1 (2014) y MQTT 3.1 (2010), siguen siendo ampliamente utilizadas.

MQTT ofrece tres niveles de calidad de servicio (QoS). El nivel QoS 0 (*fire and forget*) realiza como máximo una entrega, sin ninguna garantía de recepción. QoS 1 (*acknowledged delivery*) entrega al menos una vez, garantizando la recepción del mensaje pero con la posibilidad de recepción por duplicado. QoS 2 (*assured delivery*) entrega el mensaje exactamente una vez, asegurándose una recepción única.

El protocolo funciona en texto plano, incluso para el envío de nombres de usuario y contraseñas. Para el envío de mensajes encriptados, suele utilizarse TLS sobre TCP.

Los brokers MQTT son los servidores que reciben mensajes publicados y los distribuyen a los suscriptores. Hay implementaciones tanto comerciales (AWS IoT Core, Azure IoT Hub) como open source (Mosquitto, HiveMQ CE).

Dada su simpleza, el protocolo MQTT se utiliza en entornos de red limitados, donde la conservación de energía y ancho de banda son cruciales.

II. FRAGMENTACIÓN

Esta sección se corresponde con la resolución de los ejercicios 3 a 5.

La aplicación implementa la fragmentación del archivo en la función `publish_file` dividiendo su contenido en fragmentos de tamaño variable, determinados aleatoriamente dentro de un rango especificado. Cada fragmento se publica como un mensaje MQTT con metadatos que indican el número del fragmento, su tamaño y si es el último fragmento. Esto permite al receptor reconstruir el archivo original al recibir todos los fragmentos.

La manera en la que el script implementa la fragmentación tiene algunas similitudes con la segmentación del protocolo TCP. En el script, cada fragmento se numera (con `fragment_number`), lo que permite al receptor reconstruir los datos en el orden correcto. TCP también utiliza números de secuencia para asegurar que los segmentos se ensamblen correctamente en el destino.

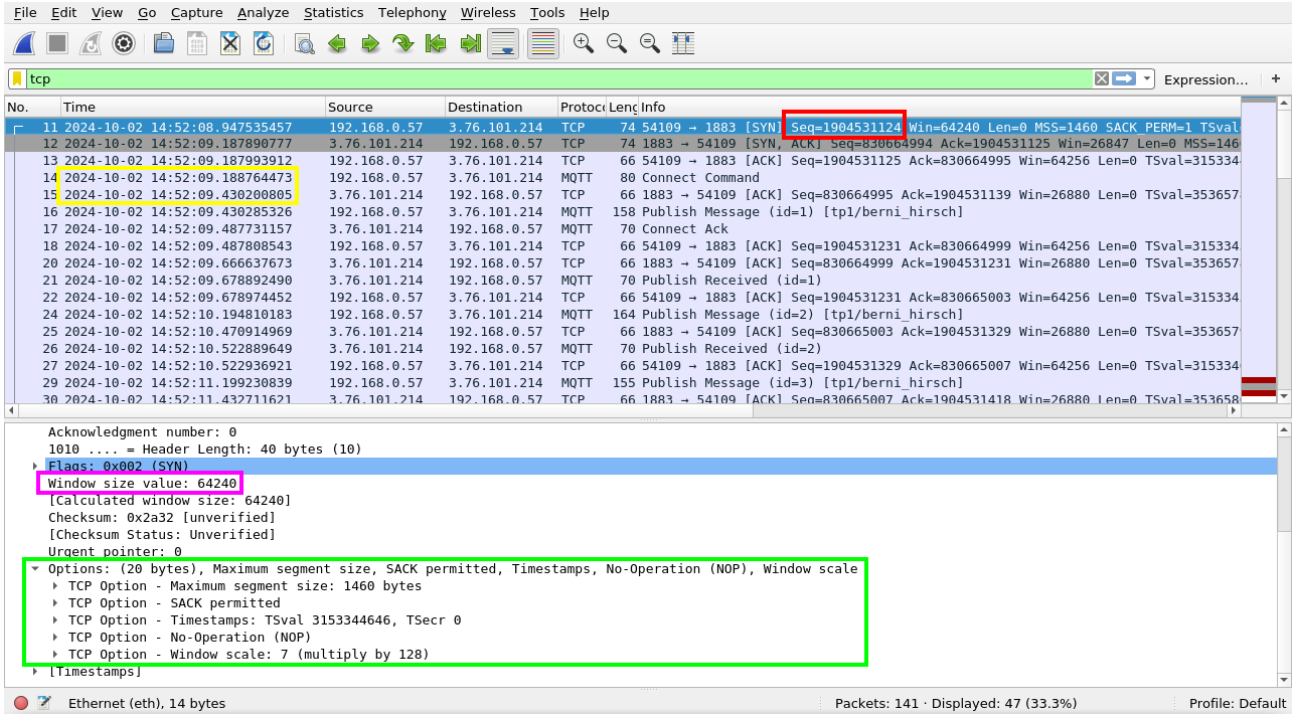

Figura 1. Captura de tráfico del publisher.

Entre las características de fragmentación que TCP tiene pero se encuentran ausentes en los scripts de MQTT está el control de flujo y congestión. El código simplemente espera un segundo (`time.sleep(1)`) entre cada envío, pero no reacciona ante posibles sobrecargas en la red o en el receptor. Tampoco hay un mecanismo de retransmisión de mensajes. Si bien TCP por debajo se asegura de que los mensajes MQTT lleguen al broker, si la transmisión del broker al subscriber fallara no hay forma de recuperar esos mensajes (sobre todo al utilizar el QoS 0 *fire and forget*).

III. Sesiones

Esta sección se corresponde con la resolución de los ejercicios 6 a 8.

En las sesiones que se establecen, tanto el publisher como el subscriber son clientes. Ambos para desemepñar su rol requieren conectarse al servidor broker, quien intermedia la comunicación. El cliente publisher se conecta al servidor broker para enviar los mensajes MQTT. El cliente subscriber también se conecta al servidor broker, en este caso para suscribirse y recibir los mensajes de un determinado tópico. Notar que la API de `paho-mqtt` explicita el rol de cliente del publisher y del subsciber a través de su clase `paho.mqtt.client.Client`.

Las sesiones TCP son persistentes, ya que en ambos casos, subscriber y publisher, hay un solo handshake antes de mandar todos los datos y un único cierre al finalizar. Cuando no hay tráfico, las conexiones se sostienen con paquetes 'keep-alive' o simplemente permanecen abiertas hasta que se alcanza un tiempo límite de inactividad.

La conexión se cierra cuando el cliente lo solicita enviando un flag FIN, lo que desencadena el four-way handshake para cerrar la conexión de manera ordenada. Cualquiera de las partes puede iniciarlo cuando ya no necesita la conexión.

III-A. Capturas del Publisher

Como se observa en la Figura 1, recuadrado en rojo, el número de secuencia del primer segmento TCP de petición de conexión es el 1904531124, cuyo ID de paquete es el 11. Se corresponde con un segmento con flag SYN, puesto que el publisher intenta establecer la conexión con el broker, para posteriormente enviar los datos correspondientes.

En la misma figura, recuadradas en verde, se encuentran las opciones que se implementan el el encabezado del segmento. La primera establece el *maximum segment size* (MSS) a 1460 bytes. La segunda indica que TCP es capaz de enviar *selective acknowledgments*, es decir, que puede enviar ACKs de paquetes recibidos fuera de orden. La tercera opción

```
Acknowledgment number: 830664995
1000 .... = Header Length: 32 bytes (8)
Flags: 0x018 (PSH, ACK)
Window size value: 502
[Calculated window size: 64256]
[Window size scaling factor: 128]
Checksum: 0x2a38 [unverified]
[Checksum Status: Unverified]
Urgent pointer: 0
Options: (12 bytes), No-Operation (NOP), No-Operation (NOP), Timestamps
[SEQ/ACK analysis]
[Timestamps]
TCP payload (14 bytes)
[PDU Size: 14]
MQ Telemetry Transport Protocol, Connect Command
```

Figura 2. Detalle del paquete ID 14.

```
Acknowledgment number: 830664995
1000 .... = Header Length: 32 bytes (8)
Flags: 0x018 (PSH, ACK)
Window size value: 502
[Calculated window size: 64256]
[Window size scaling factor: 128]
Checksum: 0x2a86 [unverified]
[Checksum Status: Unverified]
Urgent pointer: 0
Options: (12 bytes), No-Operation (NOP), No-Operation (NOP), Timestamps
[SEQ/ACK analysis]
[Timestamps]
TCP payload (92 bytes)
[PDU Size: 92]
MQ Telemetry Transport Protocol, Publish Message
```

Figura 3. Detalle del paquete ID 16.

```
Acknowledgment number: 1904531139
1000 .... = Header Length: 32 bytes (8)
Flags: 0x018 (PSH, ACK)
Window size value: 105
[Calculated window size: 26880]
[Window size scaling factor: 256]
Checksum: 0x36ac [unverified]
[Checksum Status: Unverified]
Urgent pointer: 0
Options: (12 bytes), No-Operation (NOP), No-Operation (NOP), Timestamps
[SEQ/ACK analysis]
[Timestamps]
TCP payload (4 bytes)
[PDU Size: 4]
MQ Telemetry Transport Protocol, Connect Ack
```

Figura 4. Detalle del paquete ID 17.

```
Acknowledgment number: 1904531231
1000 .... = Header Length: 32 bytes (8)
Flags: 0x018 (PSH, ACK)
Window size value: 105
[Calculated window size: 26880]
[Window size scaling factor: 256]
Checksum: 0x049d [unverified]
[Checksum Status: Unverified]
Urgent pointer: 0
Options: (12 bytes), No-Operation (NOP), No-Operation (NOP), Timestamps
[SEQ/ACK analysis]
[Timestamps]
TCP payload (4 bytes)
[PDU Size: 4]
MQ Telemetry Transport Protocol, Publish Received
```

Figura 5. Detalle del paquete ID 21.

es un timestamp que sirve como forma de ordenar los paquetes, solucionando el problema del overflow de los números de secuencia. La cuarta opción es un NOP (no option), usado para alinear las opciones a un múltiplo del ancho de palabra. Finalmente, la última opción usada es el *window scale*, que al contener un 7 indica que el tamaño de ventana reportado debe escalarse en un factor de $2^7 = 128$.

Según la información remarcada en violeta, el buffer de recepción es de 64240 bytes (window size value). Sin embargo, recordando que está la opción de window scale implementada, debe multiplicarse este valor por 128, resultando en un tamaño real de buffer de recepción de 8222720 bytes.

Como se observa en la parte recuadrada en amarillo, el primer mensaje con contenidos de aplicación (el primero de tipo MQTT, cuyo ID es 14) fue enviado a las 14:52:09.188764473, mientras que su correspondiente ACK (el segmento SYN/ACK con ID 15) fue recibido a las 14:52:09.430200805. Podemos calcular el *round trip time* (RTT) como la diferencia de estos dos tiempos, resultando en $241\,\mathrm{ms}$.

El primer segmento que transporta datos es el ID 14, tal como se mencionó, y lo envía el publisher. Su detalle se muestra en la Figura 2. Vemos recuadrado en rojo que su header tiene un tamaño de 32 bytes mientras que el payload tiene 14 bytes, resultando en un segmento de 46 bytes.

El segundo segmento con datos es el ID 16, que también se origina en el publisher. Su detalle está en la Figura 3. El header tiene nuevamente 32 bytes, mientras que el payload tiene 92 bytes, resultando en un segmento de 124 bytes.

El tercer segmento con datos es el ID 17. Este proviene del broker. Su detalle está en la Figura 4. El header tiene nuevamente 32 bytes, mientras que el payload tiene 4 bytes, resultando en un segmento de 36 bytes.

El cuarto segmento con datos es el ID 21. Este proviene del broker. Su detalle está en la Figura 5. El header tiene nuevamente 32 bytes, mientras que el payload tiene nuevamente 4 bytes, resultando en un segmento de 36 bytes.

III-B. Capturas del Subscriber

Como se puede observar en la Figura 6, recuadrado en rojo, el número de secuencia del primer segmento TCP es el 1080743669.

Por otro lado, recuadradas en azul, se pueden observar las opciones TCP. En primer lugar se encuentra el *maximum segment size*, establecido en 1460 bytes. En la segunda se observa que nuevamente se activó el *selective acknowledge-*

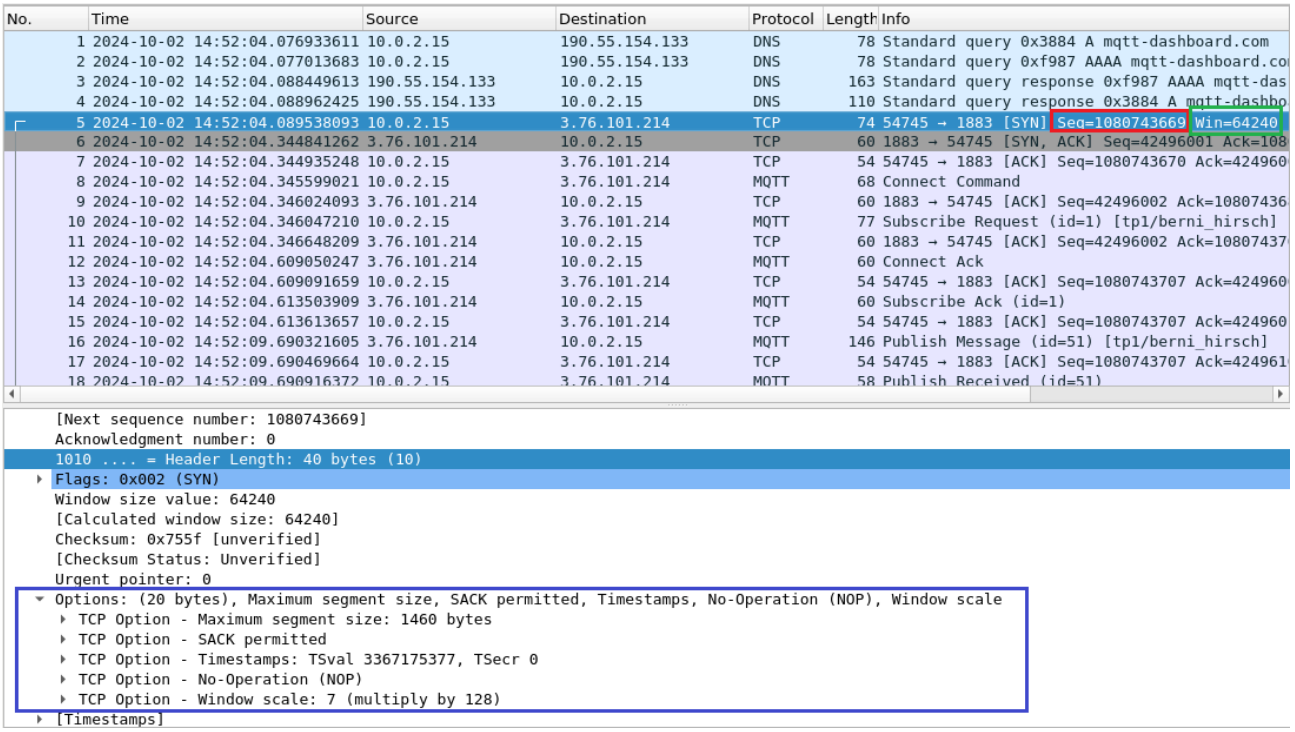

Figura 6. Captura de Wireshark del 'Subscriber'

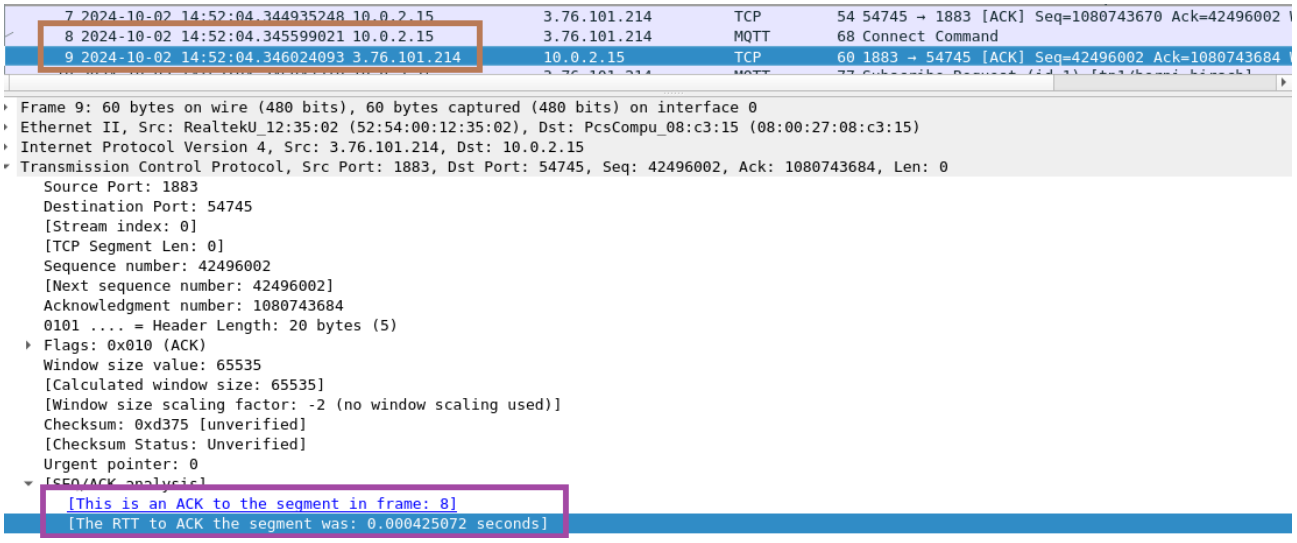

Figura 7. Captura de Wireshark del 'Subscriber'

ment. Luego figura *Timestamps*, la cual agrega una marca de tiempo a cada segmento enviado. Nuevamente se observa que las ultimas dos resultan ser *NOP* y *window scale* en 7.

Recuadrada en verde, se distingue el tamaño del buffer de recepcion, el cual es 64240 bytes, pero sufre la misma modificación que en el caso del 'Publisher' y termina siendo de 8222720 bytes.

Posteriormente, en la Figura 7 se puede observar que el primer paquete con datos de la aplicación es el de ID 8, ya que tiene el comando *Connect* correspondiente al protocolo MQTT. Este se envió a las 14:52:04.345599021. Recuadrado en violeta se observa que el paquete siguiente es el correspondiente ACK, el cual se envió a las 14:52:04.346024093.

```
  Acknowledgment number: 42496002
  0101 .... = Header Length: 20 bytes (5)
▸ Flags: 0x018 (PSH, ACK)
  Window size value: 64240
  [Calculated window size: 64240]
  [Window size scaling factor: -2 (no window scaling used)]
  Checksum: 0x7559 [unverified]
  [Checksum Status: Unverified]
  Urgent pointer: 0
▸ [SEQ/ACK analysis]
▸ [Timestamps]
  TCP payload (14 bytes)
  [PDU Size: 14]
MQ Telemetry Transport Protocol, Connect Command
```

Figura 8. Detalle del paquete ID 8.

```
  Acknowledgment number: 42496002
  0101 .... = Header Length: 20 bytes (5)
▸ Flags: 0x018 (PSH, ACK)
  Window size value: 64240
  [Calculated window size: 64240]
  [Window size scaling factor: -2 (no window scaling used)]
  Checksum: 0x7562 [unverified]
  [Checksum Status: Unverified]
  Urgent pointer: 0
▸ [SEQ/ACK analysis]
▸ [Timestamps]
  TCP payload (23 bytes)
  [PDU Size: 23]
MQ Telemetry Transport Protocol, Subscribe Request
```

Figura 9. Detalle del paquete ID 10.

```
  Acknowledgment number: 1080743707
  0101 .... = Header Length: 20 bytes (5)
▸ Flags: 0x018 (PSH, ACK)
  Window size value: 65535
  [Calculated window size: 65535]
  [Window size scaling factor: -2 (no window scaling used)]
  Checksum: 0xb350 [unverified]
  [Checksum Status: Unverified]
  Urgent pointer: 0
▸ [SEQ/ACK analysis]
▸ [Timestamps]
  TCP payload (4 bytes)
  [PDU Size: 4]
MQ Telemetry Transport Protocol, Connect Ack
```

Figura 10. Detalle del paquete ID 12.

```
  Acknowledgment number: 1080743707
  0101 .... = Header Length: 20 bytes (5)
▸ Flags: 0x018 (PSH, ACK)
  Window size value: 65535
  [Calculated window size: 65535]
  [Window size scaling factor: -2 (no window scaling used)]
  Checksum: 0x4149 [unverified]
  [Checksum Status: Unverified]
  Urgent pointer: 0
▸ [SEQ/ACK analysis]
▸ [Timestamps]
  TCP payload (5 bytes)
  [PDU Size: 5]
MQ Telemetry Transport Protocol, Subscribe Ack
```

Figura 11. Detalle del paquete ID 14.

Restando estos dos tiempos se puede obtener un RTT de $425{,}072\,\mu\text{s}$. Notar que probablemente este RTT sea incorrecto debido a un error en la captura por parte de Wireshark. Incluso suponiendo un procesamiento instantáneo de los paquetes y que los mismos viajan a la velocidad de la luz (ambas suposiciones no son correctas en la práctica), el servidor no podría estar más lejos que unos $60\,\text{km}$. Esto seguramente no se cumpla.

Luego de un análisis más profundo de la captura del subscriber, se observa que el primer segmento que transporta datos es el ID 8, el cual es enviado por el subscriber. Su detalle se muestra en la Figura 8. Vemos que el header tiene un tamaño de 20 bytes y su payload de 14 bytes, resultando en un segmento de 34 bytes.

El segundo segmento que transporta datos es el ID 10. Dicho segmento también es enviado por el subscriber y se trata de la solicitud para suscribirse a un tópico. Su detalle se muestra en la Figura 9. Su header tiene 20 bytes, mientras que su payload tiene 23 bytes, resultando en un segmento de 43 bytes.

En tercer lugar está el segmento con ID 12, en este caso enviado por el broker, aceptando la conexión solicitada por el subscriber. Su detalle se muestra en la Figura 10. Tiene un header de 20 bytes y un payload de 4 bytes, por lo tanto 24 bytes en total.

El cuarto segmento es el de ID 14, enviado por el broker, aceptando la subscripción solicitada por el subscriber. Su detalle se muestra en la Figura 11. Consta de 20 bytes de header y 5 bytes de payload, resultando en 25 bytes en total.

IV. Mejoras a los scripts

IV-A. Validación de largo

Esta subsección se corresponde con la resolución del ejercicio 9.

Para simular la pérdida de un paquete, se modificó el código del publisher de forma tal que no enviara uno de los fragmentos del texto. El script con esta modificación se llamó `publisher-drop.py`. El texto reconstruido por el subscriber quedó truncado, faltando la porción de texto perdida.. Dada la forma en que fue diseñado el código, el suscriptor no tiene forma de detectar que se ha perdido un fragmento.

Como modificación, decidimos agregar a los metadatos un campo `file_size` que indica la cantidad de bytes en el archivo. Además, se corrigió un error en el script original que enviaba un `fragment_size` incorrecto en el último fragmento. Una vez recibido el último fragmento, se compara el total de bytes recibidos con el campo `file_size` de los metadatos y, si estos difieren, se imprime por consola una advertencia. Los scripts modificados son `{publisher,subscriber}-length.py`.

IV-B. Acuse de recibo

Esta sección se corresponde con la resolución del ejercicio 10.

Con la mejora introducida anteriormente, el suscriptor es capaz de detectar la pérdida de un paquete. Sin embargo, lo único que puede hacer al respecto es advertir al usuario. Para solucionar este defecto, decidimos mejorar la comunicación a través de la implementación de un sencillo sistema de acuse de recibo y retransmisiones. El código modificado se encuentra en los archivos `{publisher,subscriber}-acks.py`.

El publisher ahora envía un máximo de 5 fragmentos consecutivos al tópico de destino `"tp1/berni_hirsch"`. Estos segmentos se añaden a un diccionario que guarda los segmentos que están pendientes de acuse de recibo. Luego, se pone a esperar mensajes al topic `"tp1/berni_hirsch_acks"`, esperando los acuses de recibo correspondientes los fragmentos enviados. Ante cada recepción de un ACK, elimina del diccionario todos los mensajes en espera de acuse de recibo que tengan un número de fragmento menor al ACK recibido. Esto le da cierta robustez ante la pérdida de algún ACK. Como TCP, esta implementación utiliza ACKs acumulativos. Si pasaran 5 segundos y hubiera aún fragmentos cuya recepción no fue confirmada, el publisher procede con su retransmisión hasta finalmente lograr un acuse de recibo. Luego, procede a enviar otra ráfaga de 5 fragmentos, repitiendo el procedimiento hasta enviar todo el archivo.

Desde el lado del subscriber, cada vez que recibe un mensaje revisa si el número de fragmento se corresponde con el número esperado. De ser así, envía un acuse de recibo para dicho fragmento y guarda la información recibida. En caso de recibir un fragmento fuera de orden, reenvía el acknowledgement con el último número de fragmento correctamente recibido y descarta el fragmento. Esto asegura que si se perdió un ACK, el publisher lo reciba. Además, garantiza la recepción en orden de los datos. Una vez recibido el último fragmento—con la bandera `is_last` encendida—envía el último ACK, reensambla el mensaje y lo guarda. Notar que por simplicidad no se implementó un cierre de la conexión ni nos aseguramos que el publisher reciba el último ACK.

Este código es capaz de enviar correctamente todo el archivo en forma completa y ordenada. Además, es capaz de reenviar fragmentos perdidos y funciona correctamente si lo que se pierde es un ACK. Entendemos que por su simplicidad pueden haber casos patológicos y de borde en los que el sistema falle, pero creemos que la consigna no es reinventar TCP sino simplemente realizar alguna mejora al sistema.